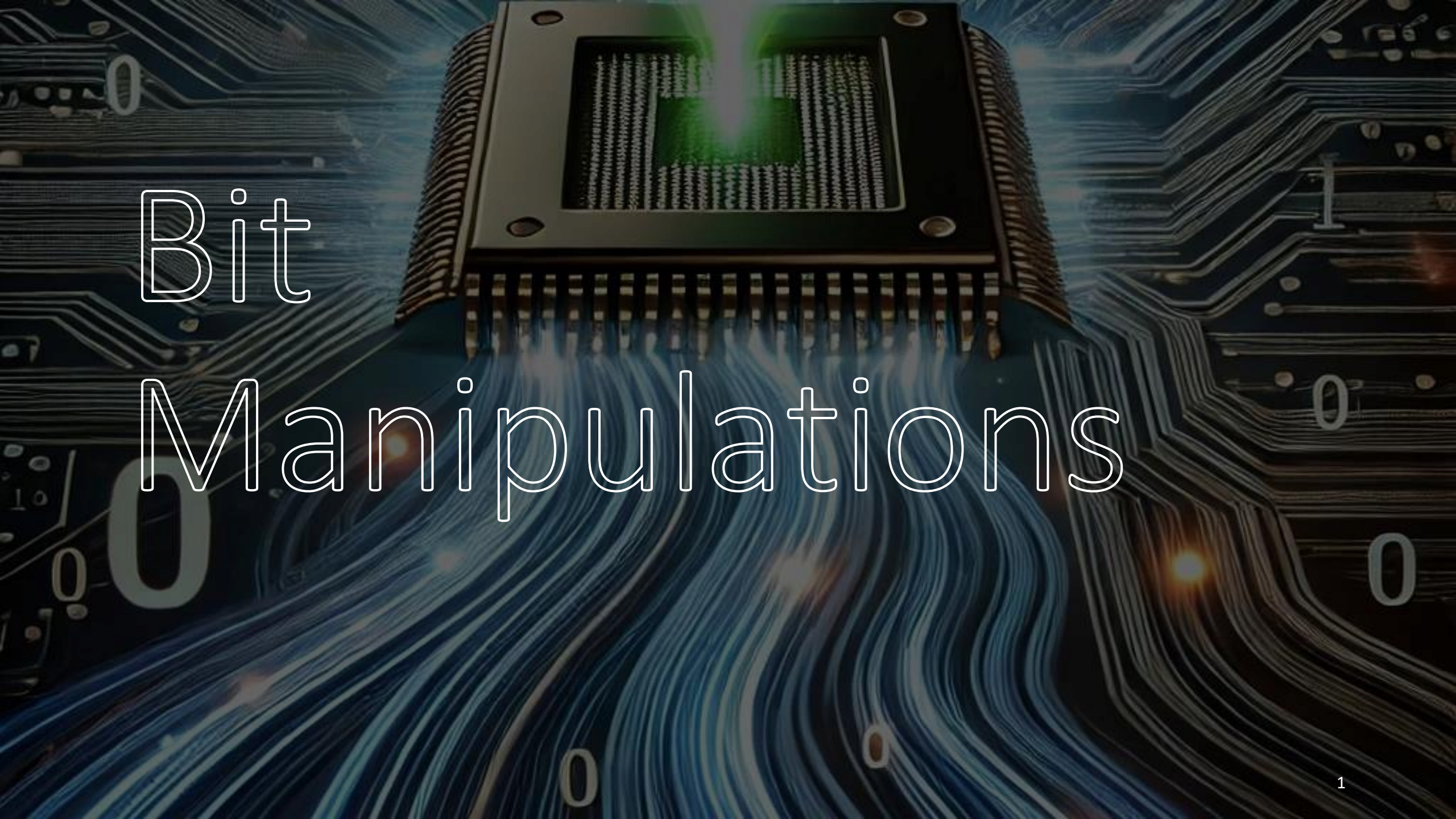




Bit Manipulations

Bitwise Operators

- All data is represented internally as sequences of bits
 - Each bit can be either 0 or 1
 - Sequence of 8 bits forms a byte
- Bitwise operators are used to manipulate the bits of integral operands
- Bitwise operations are faster than standard arithmetic
- Bitwise data manipulations are machine dependent
- Unsigned integers are normally used with the bitwise operators

Bitwise operators

Operator		Description
&	bitwise AND	The bits in the result are set to 1 if the corresponding bits in the two operands are both 1 .
	bitwise inclusive OR	The bits in the result are set to 1 if at least one of the corresponding bits in the two operands is 1 .
^	bitwise exclusive OR	The bits in the result are set to 1 if exactly one of the corresponding bits in the two operands is 1 .
<<	left shift	Shifts the bits of the first operand left by the number of bits specified by the second operand; fill from the right with 0 bits.
>>	right shift	Shifts the bits of the first operand right by the number of bits specified by the second operand; the method of filling from the left is machine dependent.
~	one's complement	All 0 bits are set to 1 and all 1 bits are set to 0 .

Results of combining two bits with the bitwise AND operator &.

Bit 1	Bit 2	Bit 1 & Bit 2
0	0	0
1	0	0
0	1	0
1	1	1

Results of combining two bits with the bitwise inclusive OR operator |.

Bit 1	Bit 2	Bit 1 Bit 2
0	0	0
1	0	1
0	1	1
1	1	1

Results of combining two bits with the bitwise exclusive OR operator $\wedge$.

Bit 1	Bit 2	Bit 1 $\wedge$ Bit 2
0	0	0
1	0	1
0	1	1
1	1	0

Example-1: Bitwise Operations

```
1  /* Fig. 10.7: fig10_07.c
2     Printing an unsigned integer in bits */
3  #include <stdio.h>
4
5  void displayBits( unsigned value ); /* prototype */
6
7  int main( void )
8  {
9     unsigned x; /* variable to hold user input */
10
11     printf( "Enter an unsigned integer: " );
12     scanf( "%u", &x );
13
14     displayBits( x );
15
16     return 0; /* indicates successful termination */
17
18 } /* end main */
19
```

Example-1: Bitwise Operations

```
20 /* display bits of an unsigned integer value */
21 void displayBits( unsigned value )
22 {
23     unsigned c; /* counter */
24
25     /* define displayMask and left shift 31 bits */
26     unsigned displayMask = 1 << 31;
27
28     printf( "%10u = ", value );
29
30     /* loop through bits */
31     for ( c = 1; c <= 32; c++ ) {
32         putchar( value & displayMask ? '1' : '0' );
33         value <<= 1; /* shift value left by 1 */
34
35         if ( c % 8 == 0 ) { /* output space after 8 bits */
36             putchar( ' ' );
37         } /* end if */
38
39     } /* end for */
40
41     putchar( '\n' );
42 } /* end function displayBits */
```

displayMask is a 1 followed by 31 zeros

Bitwise AND returns nonzero if the leftmost bits of **displayMask** and **value** are both 1, since all other bits in **displayMask** are 0s.

```
Enter an unsigned integer: 65000
65000 = 00000000 00000000 11111101 11101000
```


Example-2: Bitwise Operations

```
1  /* Fig. 10.9: fig10_09.c
2     Using the bitwise AND, bitwise inclusive OR, bitwise
3     exclusive OR and bitwise complement operators */
4  #include <stdio.h>
5
6  void displayBits( unsigned value ); /* prototype */
7
8  int main( void )
9  {
10     unsigned number1;
11     unsigned number2;
12     unsigned mask;
13     unsigned setBits;
14
15     /* demonstrate bitwise AND (&) */
16     number1 = 65535;
17     mask = 1;
18     printf( "The result of combining the following\n" );
19     displayBits( number1 );
20     displayBits( mask );
21     printf( "using the bitwise AND operator & is\n" );
22     displayBits( number1 & mask );
23
```

Bitwise AND sets each bit in the result to 1 if the corresponding bits in the operands are both 1

Example-2: Bitwise Operations

```
24  /* demonstrate bitwise inclusive OR (|) */
25  number1 = 15;
26  setBits = 241;
27  printf( "\nThe result of combining the following\n" );
28  displayBits( number1 );
29  displayBits( setBits );
30  printf( "using the bitwise inclusive OR operator | is\n" );
31  displayBits( number1 | setBits );
```

```
32
33  /* demonstrate bitwise exclusive OR (^) */
34  number1 = 139;
35  number2 = 199;
36  printf( "\nThe result of combining the following\n" );
37  displayBits( number1 );
38  displayBits( number2 );
39  printf( "using the bitwise exclusive OR operator ^ is\n" );
40  displayBits( number1 ^ number2 );
```

Bitwise inclusive OR sets each bit in the result to 1 if at least one of the corresponding bits in the operands is 1

```
41
42  /* demonstrate bitwise complement (~)*/
43  number1 = 21845;
44  printf( "\nThe one's complement of\n" );
45  displayBits( number1 );
46  printf( "is\n" );
47  displayBits( ~number1 );
```

Bitwise exclusive OR sets each bit in the result to 1 if only one of the corresponding bits in the operands is 1

```
48
49  return 0; /* indicates successful termination */
50 } /* end main */
51
```

Complement operator sets each bit in the result to 0 if the corresponding bit in the operand is 1 and vice versa

Example-2: Bitwise Operations

```
52 /* display bits of an unsigned integer value */
53 void displayBits( unsigned value )
54 {
55     unsigned c; /* counter */
56
57     /* declare displayMask and left shift 31 bits */
58     unsigned displayMask = 1 << 31;
59
60     printf( "%10u = ", value );
61
62     /* loop through bits */
63     for ( c = 1; c <= 32; c++ ) {
64         putchar( value & displayMask ? '1' : '0' );
65         value <<= 1; /* shift value left by 1 */
66
67         if ( c % 8 == 0 ) { /* output a space after 8 bits */
68             putchar( ' ' );
69         } /* end if */
70
71     } /* end for */
72
73     putchar( '\n' );
74 } /* end function displayBits */
```

Example-2: Bitwise Operations

The result of combining the following

65535 = 00000000 00000000 11111111 11111111

1 = 00000000 00000000 00000000 00000001

using the bitwise AND operator & is

1 = 00000000 00000000 00000000 00000001

The result of combining the following

15 = 00000000 00000000 00000000 00001111

241 = 00000000 00000000 00000000 11110001

using the bitwise inclusive OR operator | is

255 = 00000000 00000000 00000000 11111111

The result of combining the following

139 = 00000000 00000000 00000000 10001011

199 = 00000000 00000000 00000000 11000111

using the bitwise exclusive OR operator ^ is

76 = 00000000 00000000 00000000 01001100

The one's complement of

21845 = 00000000 00000000 01010101 01010101

is

4294945450 = 11111111 11111111 10101010 10101010

Example-3: Bitwise Operations

```
1  /* Fig. 10.13: fig10_13.c
2     Using the bitwise shift operators */
3  #include <stdio.h>
4
5  void displayBits( unsigned value ); /* prototype */
6
7  int main( void )
8  {
9     unsigned number1 = 960; /* initialize number1 */
10
11     /* demonstrate bitwise left shift */
12     printf( "\nThe result of left shifting\n" );
13     displayBits( number1 );
14     printf( "8 bit positions using the " );
15     printf( "left shift operator << is\n" );
16     displayBits( number1 << 8 );
17
```

Left shift operator shifts all bits left a specified number of spaces, filling in zeros for the empty bits

Example-3: Bitwise Operations

```
18  /* demonstrate bitwise right shift */
19  printf( "\nThe result of right shifting\n" );
20  displayBits( number1 );
21  printf( "8 bit positions using the " );
22  printf( "right shift operator >> is\n" );
23  displayBits( number1 >> 8 );
24
25  return 0; /* indicates successful termination */
26 } /* end main */
27
28 /* display bits of an unsigned integer value */
29 void displayBits( unsigned value )
30 {
30 {
31     unsigned c; /* counter */
32
33     /* declare displayMask and left shift 31 bits */
34     unsigned displayMask = 1 << 31;
35
36     printf( "%7u = ", value );
37
```

Right shift operator shifts all bits right a specified number of spaces, filling in the empty bits in an implementation-defined manner

Example-3: Bitwise Operations

```
38  /* loop through bits */
39  for ( c = 1; c <= 32; c++ ) {
40      putchar( value & displayMask ? '1' : '0' );
41      value <<= 1; /* shift value left by 1 */
42
43      if ( c % 8 == 0 ) { /* output a space after 8 bits */
44          putchar( ' ' );
45      } /* end if */
46
47  } /* end for */
48
49  putchar( '\n' );
50 } /* end function displayBits */
```

The result of left shifting

960 = 00000000 00000000 00000011 11000000

8 bit positions using the left shift operator << is

245760 = 00000000 00000011 11000000 00000000

The result of right shifting

960 = 00000000 00000000 00000011 11000000

8 bit positions using the right shift operator >> is

3 = 00000000 00000000 00000000 00000011

Checking Even or Odd

- Quick check using the least significant bit:
- Loop optimizations
- Low-level numeric routines

```
if (num & 1) printf("Odd");  
else printf("Even");
```

Bitwise operations can do **fast math operations**

```
x << 1  // multiply x by 2  
x >> 1  // divide x by 2
```


The bitwise assignment operators

Bitwise assignment operators

<code>&=</code>	Bitwise AND assignment operator.
<code> =</code>	Bitwise inclusive OR assignment operator.
<code>^=</code>	Bitwise exclusive OR assignment operator.
<code><<=</code>	Left-shift assignment operator.
<code>>>=</code>	Right-shift assignment operator.

Embedded Systems / Microcontrollers

- Used in:
- Arduino, ARM, and PIC programming
- Device I/O control

```
PORTA |= (1 << 3); // Turn ON bit 3 (e.g., LED)  
PORTA &= ~(1 << 3); // Turn OFF bit 3
```

Operator precedence and associativity

Operators	Associativity	Type
() [] . ->	left to right	highest
+ - ++ -- ! & * ~ sizeof (type)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
<< >>	left to right	shifting
< <= > >=	left to right	relational
== !=	left to right	equality
&	left to right	bitwise AND
^	left to right	bitwise OR
	left to right	bitwise OR
&&	left to right	logical AND
	left to right	logical OR
?:	right to left	conditional
= += -= *= /= &= = ^= <<= >>= %=	right to left	assignment
,	left to right	comma

Bit Fields

- Bit field
 - Member of a structure whose size (in bits) has been specified
 - Enable better memory utilization
 - Must be defined as `int` or `unsigned`
 - Cannot access individual bits
- Defining bit fields
 - Follow `unsigned` or `int` member with a colon (:) and an integer constant representing the width of the field
 - Example:

```
struct BitCard {  
    unsigned face : 4;  
    unsigned suit : 2;  
    unsigned color : 1;  
};
```

Example-1 : Bit Fields

```
1  /* Fig. 10.16: fig10_16.c
2     Representing cards with bit fields in a struct */
3
4  #include <stdio.h>
5
6  /* bitCard structure definition with bit fields */
7  struct bitCard {
8      unsigned face : 4; /* 4 bits; 0-15 */
9      unsigned suit : 2; /* 2 bits; 0-3 */
10     unsigned color : 1; /* 1 bit; 0-1 */
11 }; /* end struct bitCard */
12
13 typedef struct bitCard Card; /* new type name for struct bitCard */
14
15 void fillDeck( Card * const wDeck ); /* prototype */
16 void deal( const Card * const wDeck ); /* prototype */
17
18 int main( void )
19 {
20     Card deck[ 52 ]; /* create array of Cards */
21
22     fillDeck( deck );
23     deal( deck );
24
25     return 0; /* indicates successful termination */
26
27 } /* end main */
28
```

Bit fields determine how much memory
each member of a structure can take up

Example-1 : Bit Fields

```
29 /* initialize Cards */
30 void fillDeck( Card * const wDeck )
31 {
32     int i; /* counter */
33
34     /* loop through wDeck */
35     for ( i = 0; i <= 51; i++ ) {
36         wDeck[ i ].face = i % 13;
37         wDeck[ i ].suit = i / 13;
38         wDeck[ i ].color = i / 26;
39     } /* end for */
40
41 } /* end function fillDeck */
42
43 /* output cards in two column format; cards 0-25 subscripted with
44    k1 (column 1); cards 26-51 subscripted k2 (column 2) */
45 void deal( const Card * const wDeck )
46 {
47     int k1; /* subscripts 0-25 */
48     int k2; /* subscripts 26-51 */
49
50     /* loop through wDeck */
51     for ( k1 = 0, k2 = k1 + 26; k1 <= 25; k1++, k2++ ) {
52         printf( "Card:%3d  Suit:%2d  Color:%2d  ",
53             wDeck[ k1 ].face, wDeck[ k1 ].suit, wDeck[ k1 ].color );
54         printf( "Card:%3d  Suit:%2d  Color:%2d\n",
55             wDeck[ k2 ].face, wDeck[ k2 ].suit, wDeck[ k2 ].color );
56     } /* end for */
57
58 } /* end function deal */
```

Example-1 : Bit Fields

Card: 0	Suit: 0	Color: 0	Card: 0	Suit: 2	Color: 1
Card: 1	Suit: 0	Color: 0	Card: 1	Suit: 2	Color: 1
Card: 2	Suit: 0	Color: 0	Card: 2	Suit: 2	Color: 1
Card: 3	Suit: 0	Color: 0	Card: 3	Suit: 2	Color: 1
Card: 4	Suit: 0	Color: 0	Card: 4	Suit: 2	Color: 1
Card: 5	Suit: 0	Color: 0	Card: 5	Suit: 2	Color: 1
Card: 6	Suit: 0	Color: 0	Card: 6	Suit: 2	Color: 1
Card: 7	Suit: 0	Color: 0	Card: 7	Suit: 2	Color: 1
Card: 8	Suit: 0	Color: 0	Card: 8	Suit: 2	Color: 1
Card: 9	Suit: 0	Color: 0	Card: 9	Suit: 2	Color: 1
Card: 10	Suit: 0	Color: 0	Card: 10	Suit: 2	Color: 1
Card: 11	Suit: 0	Color: 0	Card: 11	Suit: 2	Color: 1
Card: 12	Suit: 0	Color: 0	Card: 12	Suit: 2	Color: 1
Card: 0	Suit: 1	Color: 0	Card: 0	Suit: 3	Color: 1
Card: 1	Suit: 1	Color: 0	Card: 1	Suit: 3	Color: 1
Card: 2	Suit: 1	Color: 0	Card: 2	Suit: 3	Color: 1
Card: 3	Suit: 1	Color: 0	Card: 3	Suit: 3	Color: 1
Card: 4	Suit: 1	Color: 0	Card: 4	Suit: 3	Color: 1
Card: 5	Suit: 1	Color: 0	Card: 5	Suit: 3	Color: 1
Card: 6	Suit: 1	Color: 0	Card: 6	Suit: 3	Color: 1
Card: 7	Suit: 1	Color: 0	Card: 7	Suit: 3	Color: 1
Card: 8	Suit: 1	Color: 0	Card: 8	Suit: 3	Color: 1
Card: 9	Suit: 1	Color: 0	Card: 9	Suit: 3	Color: 1
Card: 10	Suit: 1	Color: 0	Card: 10	Suit: 3	Color: 1
Card: 11	Suit: 1	Color: 0	Card: 11	Suit: 3	Color: 1
Card: 12	Suit: 1	Color: 0	Card: 12	Suit: 3	Color: 1